

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Danny Enrique Balladares Cherres

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO ABRIL 2026 - AGOSTO 2026



Actividades a desarrollar en la Unidad 1

Actividad # Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

1. Análisis del problema

Según Rodríguez Gallardo et al. (2018), los sistemas Help Desk son herramientas fundamentales para la gestión eficiente de incidentes en organizaciones. En este sentido, el análisis del problema se enfoca en identificar las necesidades críticas de un sistema de gestión de incidentes que permita registrar, clasificar, asignar y resolver solicitudes de soporte técnico de manera ordenada y eficiente.

Como señalan Serrano Preciado et al. (2024), la implementación de estos sistemas es esencial para mejorar la calidad del servicio y la satisfacción de los usuarios. Por consiguiente, es necesario desarrollar una solución que integre funcionalidades robustas y un diseño escalable que responda a los requerimientos operacionales y técnicos de la organización.

1.1 Autores Principales

Los actores principales del sistema de gestión de incidentes fueron identificados mediante un análisis exhaustivo de los requisitos operacionales y las mejores prácticas en sistemas Help Desk. Conforme a lo expuesto por Rodríguez Gallardo, López de la Madrid y Espinoza de los Monteros (2018), la implementación de software Help Desk en instituciones requiere la participación de múltiples actores con responsabilidades diferenciadas. En primer lugar, el Usuario Final actúa como reportante de incidentes, siendo responsable de crear y documentar solicitudes técnicas, así como realizar seguimiento de sus tickets.

En segundo lugar, el Técnico de Soporte desempeña un papel fundamental como resolutor de incidentes, encargándose de recibir, diagnosticar y ejecutar soluciones a los problemas reportados. Asimismo, Serrano Preciado, Valencia Moreno, Osorio Cayetano y Wagner Gutiérrez (2024) enfatizan la importancia del Administrador del Sistema como gestor general del ambiente, quien supervisa el desempeño, gestiona usuarios, genera reportes y realiza auditorías de seguridad. Por último, Pérez Romero et al. (2024) subrayan que la correcta definición de roles y responsabilidades es esencial para consolidar un sistema Help Desk funcional y eficiente, asegurando así que cada actor contribuya de manera óptima al logro de los objetivos organizacionales.



Actor	Rol	Responsabilidades	Permisos
Usuario Final	Reportante de Incidentes	Crear y reportar incidentes técnicos; proporcionar detalles sobre problemas; realizar seguimiento del estado de sus solicitudes	Crear incidentes, ver estado de sus propios tickets, añadir comentarios a sus incidentes
Técnico de Soporte	Resolutor de Incidentes	Recibir y revisar incidentes asignados; diagnosticar problemas; ejecutar soluciones; actualizar el estado de los tickets; documentar acciones realizadas	Asignar incidentes a sí mismo, actualizar estados, cerrar incidentes, ver todos los incidentes asignados, añadir notas técnicas
Administrador del Sistema	Gestor General	Supervisar el desempeño del sistema; gestionar usuarios y permisos; generar reportes; configurar categorías y prioridades; auditar actividades; realizar mantenimiento	Acceso total al sistema, crear/editar/eliminar usuarios, generar reportes, configurar parámetros del sistema, asignar incidentes, acceso a logs de auditoría

1.2 Requerimientos Funcionales

Los requerimientos funcionales representan las capacidades específicas que el sistema debe ejecutar para cumplir con los objetivos organizacionales. De acuerdo con Rodríguez Barajas (2017), los requerimientos son elementos fundamentales que impactan directamente en la calidad del software desarrollado. Por lo tanto, estos requerimientos definen las acciones concretas que el sistema debe realizar, tales como la creación, lectura, actualización y eliminación de incidentes. A continuación, se presenta la tabla con los requerimientos funcionales identificados:

ID	REQUERIMIENTO FUNCIONAL	DESCRIPCIÓN
RF-001	Crear Incidente	El sistema debe permitir a los usuarios registrar nuevos incidentes con información detallada
RF-002	Asignar Incidente	El sistema debe asignar automáticamente o manualmente incidentes a técnicos disponibles
RF-003	Actualizar Estado	El sistema debe permitir cambiar el estado del incidente (Abierto, En Progreso, Resuelto, Cerrado)
RF-004	Visualizar Dashboard	El sistema debe mostrar un panel de control con métricas e indicadores de desempeño
RF-005	Generar Reportes	El sistema debe generar reportes de incidentes por período, categoría y técnico
RF-006	Autenticación de Usuarios	El sistema debe validar credenciales y controlar acceso según perfiles
RF-007	Notificaciones	El sistema debe enviar notificaciones sobre cambios de estado de incidentes



1.3 Requerimientos No Funcionales

Los requerimientos no funcionales especifican las características de calidad, rendimiento y seguridad que el sistema debe cumplir. Conforme a Pérez Romero et al. (2024), estos requerimientos son igualmente críticos para garantizar que el sistema funcione de manera eficiente y segura en el ambiente de producción. Por consiguiente, estos requerimientos abarcan aspectos como la disponibilidad, la seguridad de datos, el rendimiento y la usabilidad del sistema. Se presenta la siguiente tabla con los requerimientos no funcionales identificados:

ID	REQUERIMIENTO NO FUNCIONAL	DESCRIPCIÓN
RNF-001	Disponibilidad	El sistema debe estar disponible el 99% del tiempo
RNF-002	Tiempo de Respuesta	Las consultas deben responderse en menos de 2 segundos
RNF-003	Seguridad	Los datos deben estar encriptados en tránsito y en reposo
RNF-004	Escalabilidad	El sistema debe soportar hasta 1000 usuarios simultáneos
RNF-005	Compatibilidad	Debe funcionar en navegadores Chrome, Firefox, Safari y Edge
RNF-006	Mantenibilidad	El código debe seguir estándares de documentación y buenas prácticas
RNF-007	Recuperación	El sistema debe contar con mecanismo de backup diario

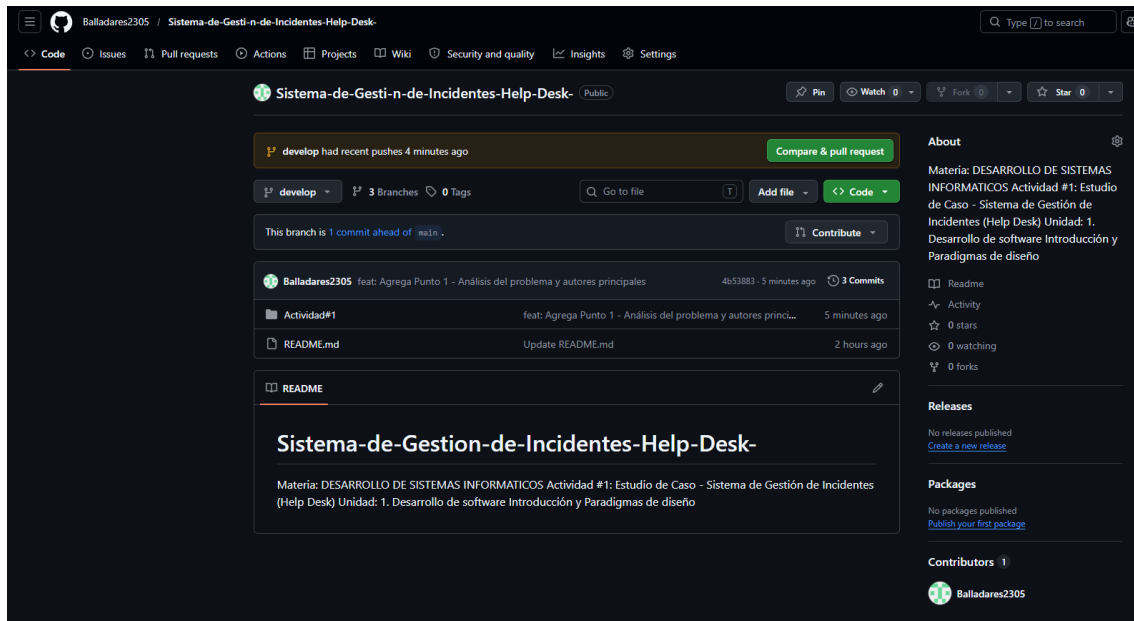
2. Diseño del sistema

El diseño del sistema de gestión de incidentes se fundamenta en una arquitectura modular que integra patrones de diseño reconocidos y mejores prácticas de ingeniería de software. De acuerdo con los principios establecidos por Serrano Preciado et al. (2024), el desarrollo de un sistema Help Desk requiere un enfoque estructurado que combine claridad en la especificación de componentes, flexibilidad en la implementación y escalabilidad para futuras extensiones.

Para esto, se utiliza un repositorio centralizado con control de versiones mediante Gitflow, que permite la colaboración ordenada entre desarrolladores. Además, se implementan diagramas UML que visualizan la interacción entre componentes, casos de uso que representan las funcionalidades del sistema, y una arquitectura general que define cómo se integran los módulos de backend, frontend y base de datos. Este enfoque integral garantiza que el sistema sea robusto, mantenible y capaz de adaptarse a los requisitos cambiantes de la organización.

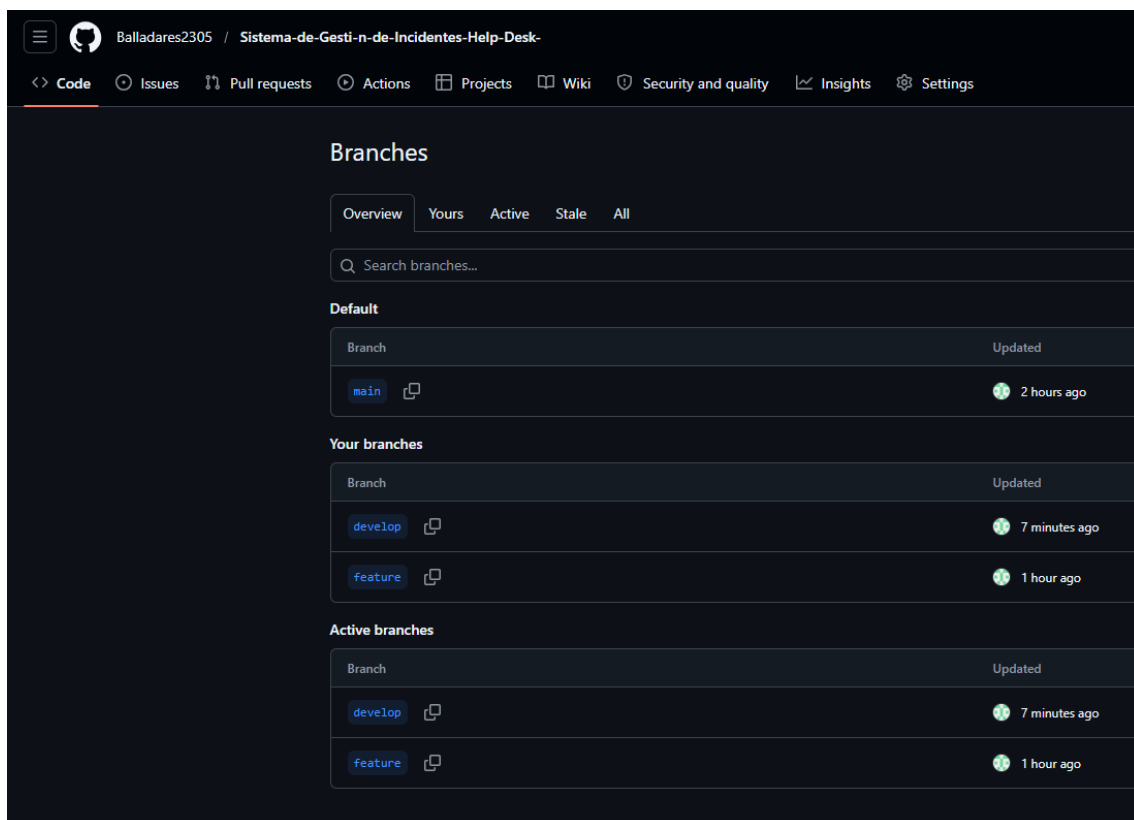


2.1 Repositorio



Se creo el repositorio en GitHub

2.2 Gitflow



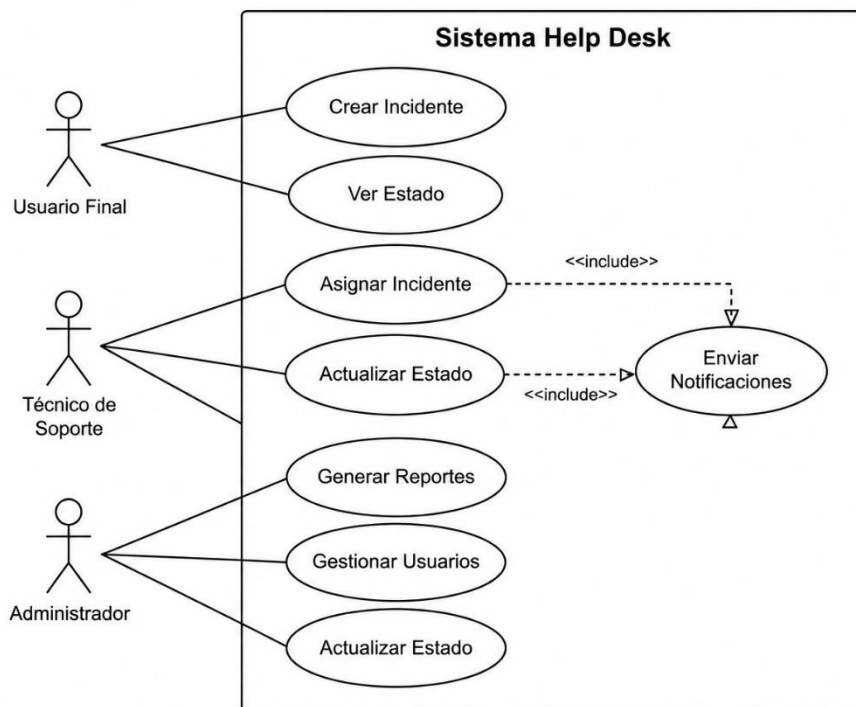
Se crearon las diferentes ramas

Gitflow es un modelo de ramificación que establece un flujo de trabajo estructurado para el desarrollo colaborativo. Como señala Atlassian (s.f.), este enfoque define ramas específicas para desarrollo, lanzamiento y producción, permitiendo que

múltiples desarrolladores trabajen simultáneamente sin conflictos. El flujo incluye la rama main (producción), develop (integración), feature/* (nuevas funcionalidades), release/* (preparación de versiones) y hotfix/* (correcciones urgentes). Esta estructura garantiza que el código en producción sea siempre estable y que los cambios sean revisados antes de su integración.

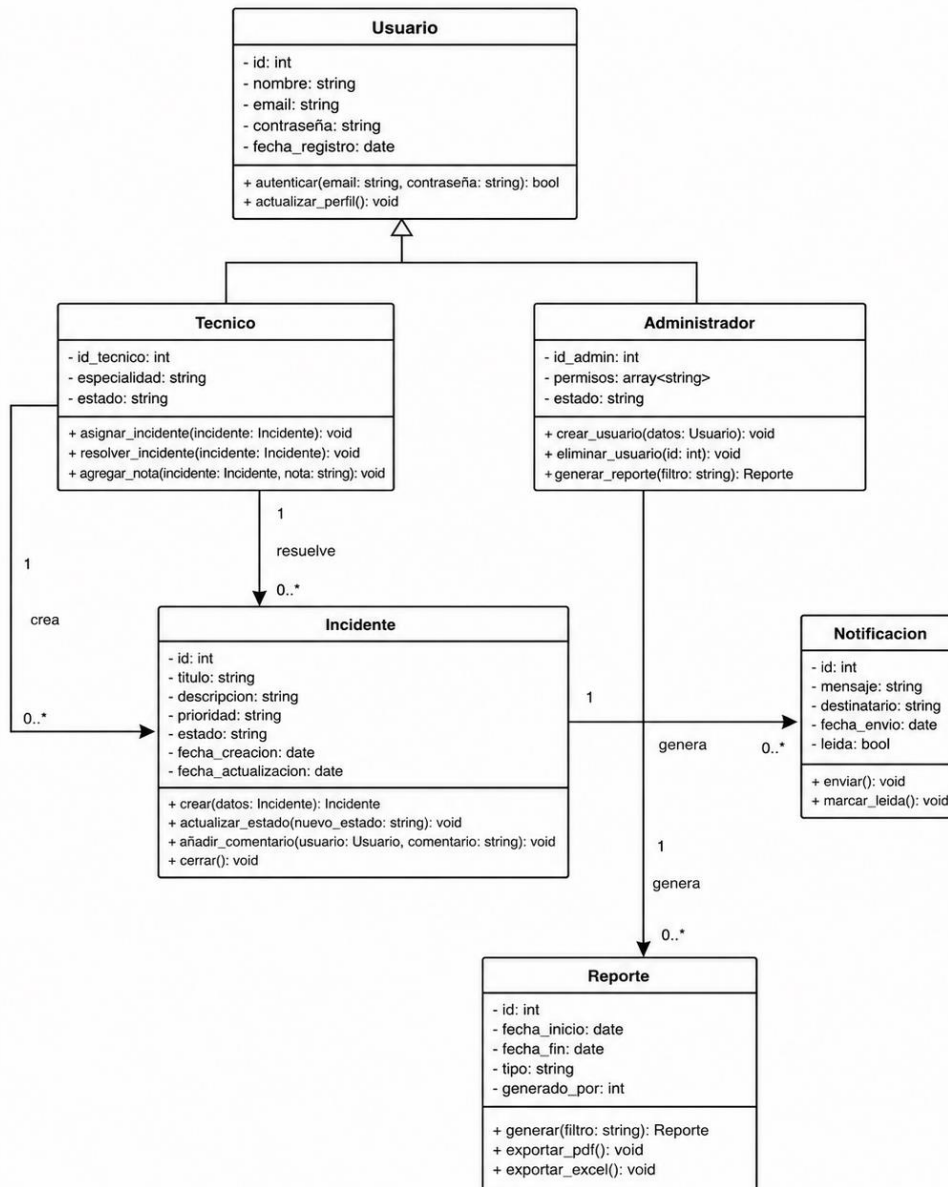
2.3 Diagrama de casos de uso

El diagrama de casos de uso representa las interacciones entre los actores del sistema (Usuario Final, Técnico de Soporte, Administrador) y las funcionalidades principales del sistema Help Desk. Este diagrama ilustra los procesos clave como crear incidentes, asignar tickets, actualizar estados, generar reportes y gestionar usuarios. Los casos de uso permiten comprender visualmente qué funcionalidades disponibles tiene cada actor y cómo interactúan con el sistema.



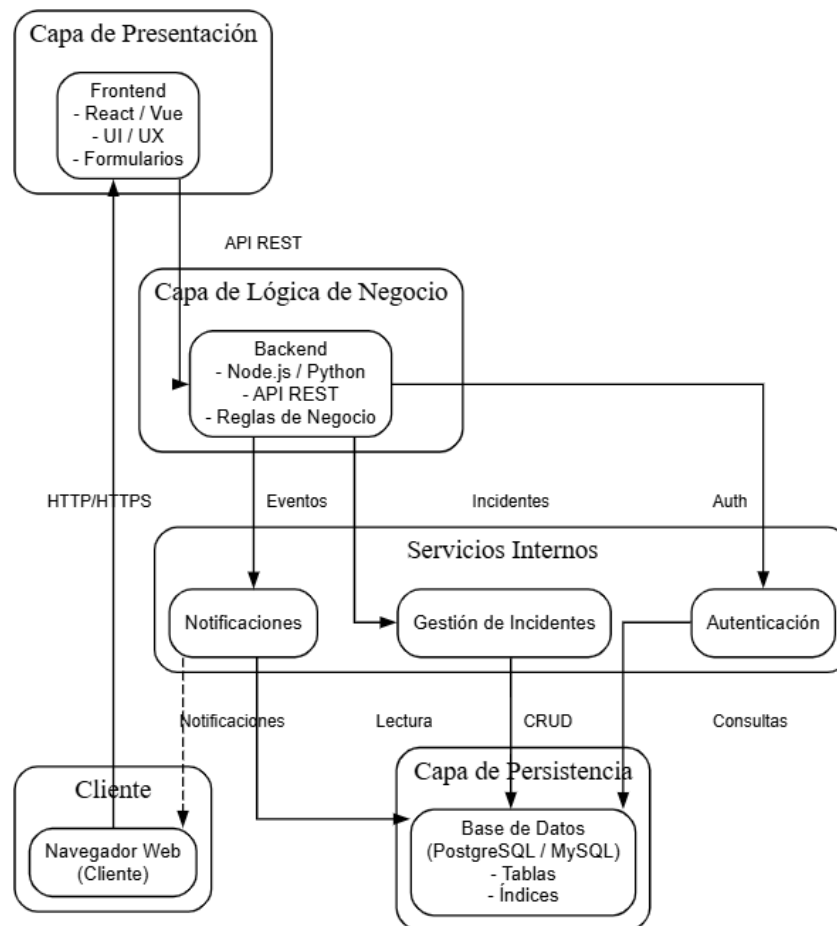
2.4 Diagrama de clases UML

El diagrama de clases UML especifica la estructura orientada a objetos del sistema, definiendo las clases principales, sus atributos, métodos y relaciones. Este diagrama incluye las clases Incidente, Usuario, Tecnico, Administrador, Notificacion y Reporte, mostrando cómo se relacionan entre sí mediante herencia, composición y asociación. La claridad en la estructura de clases facilita la implementación del código y asegura una arquitectura coherente.



2.5 Arquitectura general del sistema

La arquitectura general del sistema sigue un patrón de tres capas: presentación (frontend), lógica de negocio (backend) y persistencia (base de datos). Esta separación de responsabilidades permite que cada capa sea desarrollada, testada y mantenida de forma independiente. La capa de presentación proporciona la interfaz de usuario, la capa de lógica implementa las reglas de negocio y la gestión de incidentes, y la capa de datos almacena toda la información del sistema. La comunicación entre capas se realiza mediante APIs REST, asegurando un acoplamiento débil y una escalabilidad mejorada.



3. Aplicación de patrones de diseño

3.1 Singleton

El patrón Singleton es un patrón de creación que garantiza que una clase tenga una única instancia en toda la aplicación y proporciona un punto de acceso global a esa instancia. Conforme a lo expuesto por Pérez Romero et al. (2024), los patrones de diseño son fundamentales para crear soluciones robustas y mantenibles. En el contexto del sistema de gestión de incidentes, el patrón Singleton se aplicará a la clase ConfiguraciónSistema, que almacena parámetros globales como la conexión a la base de datos, configuraciones de correo electrónico y niveles de log.

De esta manera, se asegura que exista una única instancia de configuración en toda la aplicación, evitando inconsistencias y mejorando el rendimiento al no duplicar recursos críticos. Además, el patrón Singleton se puede aplicar a la clase GestorNotificaciones, permitiendo que todos los módulos del sistema accedan al mismo gestor de notificaciones de forma centralizada.



3.2 Observer (Publicador-Suscriptor)

El patrón Observer, también conocido como Publicador-Suscriptor, establece una relación de uno-a-muchos entre objetos, donde cuando un objeto (sujeto) cambia de estado, todos sus observadores (suscriptores) son notificados automáticamente. Según Serrano Preciado et al. (2024), este patrón es esencial para crear sistemas desacoplados donde múltiples componentes pueden reaccionar a eventos sin estar directamente vinculados.

En el sistema Help Desk, el patrón Observer se aplicará a la clase Incidente, que actúa como sujeto. Cuando un incidente cambia de estado (por ejemplo, de "Abierto" a "En Progreso"), todos los observadores registrados (como NotificadorUsuario, ActualizadorDashboard y RegistradorAuditoria) serán notificados automáticamente. Esta implementación permite que múltiples sistemas reaccionen a cambios sin que la clase Incidente necesite conocer los detalles de cada observador, mejorando la flexibilidad y mantenibilidad del código.

3.3 Factory Method

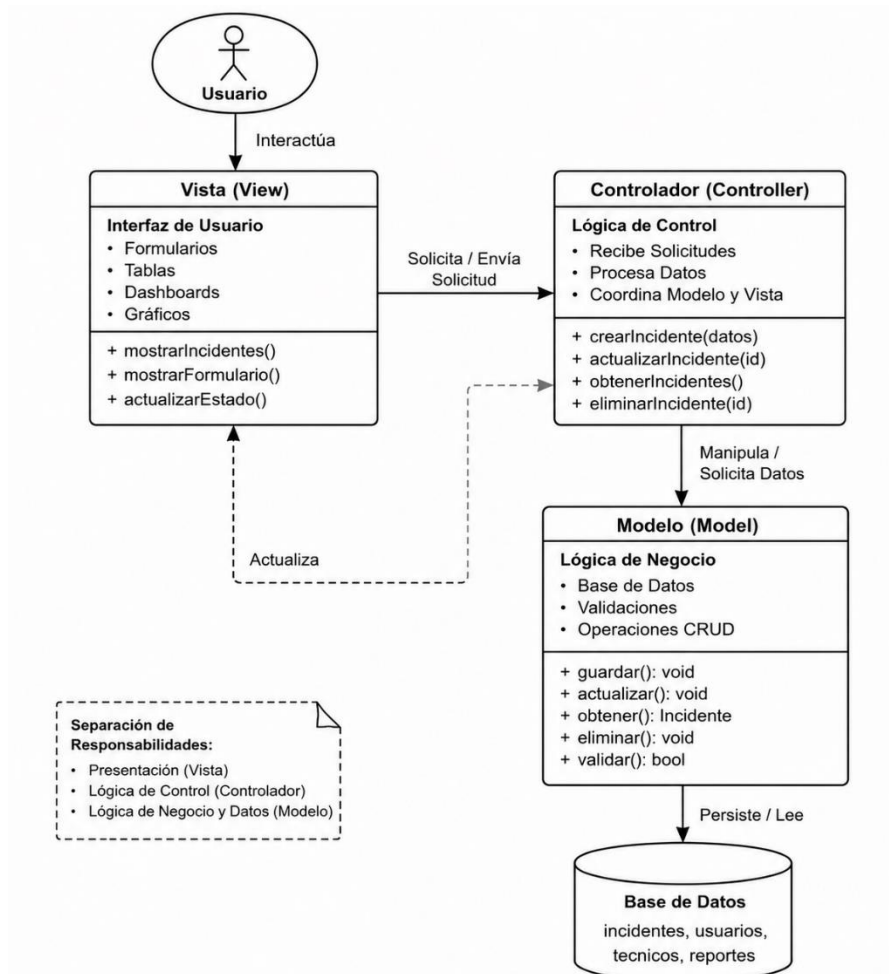
El patrón Factory Method es un patrón de creación que proporciona una interfaz para crear objetos sin especificar sus clases concretas. De acuerdo con Rodríguez Barajas (2017), los patrones de diseño mejoran la calidad del software al reducir el acoplamiento entre componentes. En el sistema Help Desk, el patrón Factory Method se aplicará a través de una clase FabricaIncidentes que es responsable de crear diferentes tipos de incidentes (Incidente Técnico, Incidente de Hardware, Incidente de Software, Incidente de Red).

En lugar de que el cliente instancie directamente cada tipo de incidente, la fábrica se encarga de esta tarea, permitiendo cambiar la lógica de creación sin afectar el código cliente. Esta implementación facilita la extensión del sistema para agregar nuevos tipos de incidentes en el futuro, mejorando la mantenibilidad y reduciendo la duplicación de código.

3.4 MVC (Model-View-Controller)

El patrón MVC es un patrón arquitectónico que separa la aplicación en tres componentes interconectados: Model (Modelo), View (Vista) y Controller (Controlador). Conforme a lo señalado por Pérez Romero et al. (2024), la arquitectura MVC es fundamental para desarrollar aplicaciones mantenibles y escalables, permitiendo que diferentes desarrolladores trabajen en diferentes componentes simultáneamente.

En el sistema Help Desk, el patrón MVC se aplicará de la siguiente manera: el Modelo contiene la lógica de negocio y el acceso a datos (clases como ModeloIncidente, ModeloUsuario, ModeloTecnico), la Vista presenta la interfaz de usuario (formularios, tablas, dashboards), y el Controlador gestiona la interacción entre el Modelo y la Vista (recibiendo solicitudes, procesando datos y actualizando la Vista). Esta separación permite que cambios en la presentación no afecten la lógica de negocio, facilitando el testing y el mantenimiento del código.



4. Justificación del Uso de Gitflow

La implementación de Gitflow en el desarrollo del sistema de gestión de incidentes es una decisión estratégica fundamental que responde a las necesidades de un proyecto colaborativo y de mediano-largo plazo. En primer lugar, Gitflow proporciona una estructura clara y ordenada para el trabajo en equipo, permitiendo que múltiples desarrolladores trabajen simultáneamente en diferentes funcionalidades sin generar conflictos o inconsistencias en el código. Esta separación de ramas (feature, develop, release, hotfix) facilita que cada miembro del equipo tenga su propio espacio de trabajo



aislado, lo cual mejora la productividad y reduce significativamente los errores de integración.

En segundo lugar, Gitflow garantiza la estabilidad del código en producción mediante un proceso de validación riguroso antes de desplegar cambios. La rama main se mantiene siempre en un estado estable y funcionalmente probado, mientras que la rama develop actúa como punto de integración para todas las nuevas funcionalidades. Esto significa que cualquier problema detectado durante el desarrollo no afecta a los usuarios finales, asegurando una experiencia de usuario consistente y confiable. Además, el flujo permite crear ramas release donde se pueden realizar pruebas finales, ajustes menores y preparación de documentación antes del lanzamiento oficial.

5. Evidencias

```
MINGW64~/c:/Users/israe/Sistema-de-Gesti-n-de-Incidentes-Help-Desk-
From https://github.com/Balladares2305/Sistema-de-Gesti-n-de-Incidentes-Help-Desk-
* branch      develop    -> FETCH_HEAD
Already up to date.

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git add Actividad#1/Actividad_1_Danny_Balladares.docx

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git commit -m "feat: Agrega Punto 1 - Análisis del problema y autores principales"
[develop 4b53883] feat: Agrega Punto 1 - Análisis del problema y autores principales
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Actividad#1/Actividad_1_Danny_Balladares.docx

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git push origin develop
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.42 MiB | 855.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Balladares2305/Sistema-de-Gesti-n-de-Incidentes-Help-Desk-.git
   ea63cd1..4b53883  develop -> develop

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$
```

Subida de análisis del problema



```
PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git add Actividad#1/diseño/

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git commit -m "feat: Agrega Punto 2 - Diagramas de diseño (casos de uso, clases, arquitectura)"
[develop 7c4648e] feat: Agrega Punto 2 - Diagramas de diseño (casos de uso, clases, arquitectura)
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 "Actividad#1/dise\303\261o/diagrama_UML.png"
create mode 100644 "Actividad#1/dise\303\261o/diagrama_caso_de_uso.png"
create mode 100644 "Actividad#1/dise\303\261o/diagrama_de_arquitectura.png"

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git push origin develop
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.86 MiB | 1.22 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Balladares2305/Sistema-de-Gesti-n-de-Incidentes-Help-Desk-.git
4b53883..7c4648e develop -> develop

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$
```

Subida de los Diagramas de Diseño

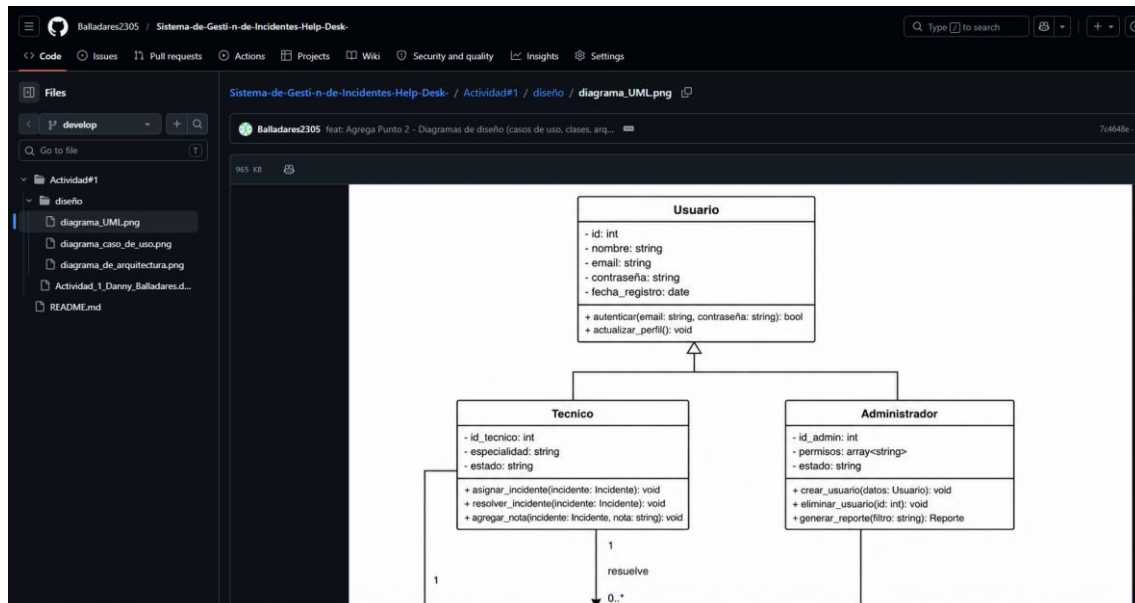
```
PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git add Actividad#1/Actividad_1_Danny_Balladares.docx

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git commit -m "feat: Agrega Punto 3 - Aplicación de patrones de diseño (Singleton, Observer, Factory, MVC)"
[develop 58a03dd] feat: Agrega Punto 3 - Aplicación de patrones de diseño (Singleton, Observer, Factory, MVC)
1 file changed, 0 insertions(+), 0 deletions(-)

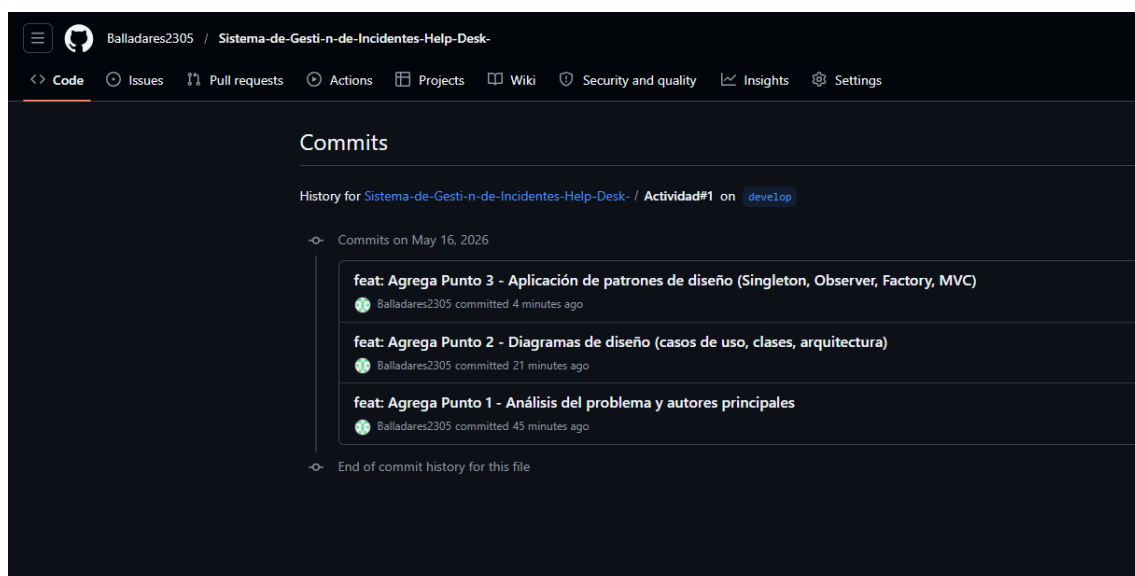
PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$ git push origin develop
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 3.37 MiB | 1.60 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Balladares2305/Sistema-de-Gesti-n-de-Incidentes-Help-Desk-.git
7c4648e..58a03dd develop -> develop

PC_TIC@_PC_TIC MINGW64 ~/Sistema-de-Gesti-n-de-Incidentes-Help-Desk- (develop)
$
```

Subida de Avance de los patrones de diseño



Estructura de Carpeta en GitHub



Historial de Commits

6. Enlace del Proyecto en GitHub

<https://github.com/Balladares2305/Sistema-de-Gesti-n-de-Incidentes-Help-Desk-.git>

Bibliografía

Atlassian. (s. f.). Flujo de trabajo de Gitflow | Tutorial de Atlassian sobre Git.

<https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow>

Pérez Romero, S. et al. (2024). Sistema Help Desk: una solución para la gestión de incidencias. Revista Gacien, Universidad Nacional Hermilio Valdizán.

<https://doi.org/10.46794/gacien.10.2.2188>



- Rodríguez Barajas, C. T. (2017). Impacto de los requerimientos en la calidad de software. *Tecnología Investigación y Academia*, 5(2), 161–173. <https://revistas.udistrital.edu.co/index.php/tia/article/view/7607>
- Rodríguez Gallardo, Juan Armando, López de la Madrid, María Cristina, & Espinoza de los Monteros Cárdenas, Adolfo. (2018). Estudio sobre la implementación del software Help Desk en una institución de educación superior. *PAAKAT: revista de tecnología y sociedad*, 8(14), 00003. Epub 01 de agosto de 2018. <https://doi.org/10.32870/pk.a8n14.298>
- Serrano Preciado, O. A., Valencia Moreno, J. M., Osorio Cayetano, O. R., & Wagner Gutiérrez, J. M. (2024). Sistema para la gestión de incidencias de cómputo en una universidad pública. *Revista De Investigación En Tecnologías De La Información*, 12(26), 179–193. <https://doi.org/10.36825/RITI.12.26.014>